Using Neuroevolution for Intrusion Detection
System Requirements Specification
Version <1.0>
10/1/2024

Document Control

Distribution List

The following list of people will receive a copy of this document every time a new version of this document becomes available:

Teaching assistants:

Jose Alejandro Gonzalez Nunez

Customer(s): Dr. M. Ilhan Akbas

Project team members:

Regan Bossie, Tyler Donay, Megan Krajewski, Richard Pepe

Change Summary

The following table details changes made between versions of this document:

Version	Date	Modifier	Description
1.0	10/29/2024		Initial release of SRS

Table of Contents

Document Control ii
Distribution List ii
Change Summary ii
1. Introduction 1
1.1. Purpose and Scope 1
1.2. Intended Audience and Reading Suggestions 2
1.3. Document Conventions 2
1.4. Project References 2
1.5. Definitions, Acronyms, and Abbreviations 2
1.5.1. Definitions 2
1.5.2. Acronyms 3
1.5.3. Table 2: Acronyms 3
1.5.4. Abbreviations 5
2. General Description 6
2.1. Product Perspective 6
2.2. Product Features 6
2.3. User Classes and Characteristics 7
2.3.1. Actors 8
2.3.2. Use Cases 8
2.3.3. Scenarios 8
2.4. General Constraints 11
2.5. Operating Environment 11
2.6. User Documentation 11
2.7. Assumptions and Dependencies 11
3. External Interface Requirements 13
3.1. User Interfaces 13
3.2. Hardware Interfaces 13
3.3. Software Interfaces 13
3.4. Communications Interfaces 14
4. Behavioral Requirements 16
4.1. Same Class of User 16
4.2. Related Real-world Objects 16
4.3. Stimulus 16

4.4. Related Features 16
4.5. Functional 16
5. Non-behavioral Requirements 18
5.1. Performance Requirements 18
5.2. Safety Requirements 18
5.3. Qualitative Requirements 18
5.3.1. Availability 18
5.3.2. Security 18
5.3.3. Maintainability 18
5.3.4. Portability 19
5.4. Design and Implementation Constraints 19
6. Other Requirements 20
6.1. Database Requirements 20
6.2. Operations 20
7. Analysis Models 22
7.1. Data Flow Model 22
7.1.1. Data Sources 22
7.1.2. Data Sinks 22
7.1.3. Data Dictionary 22
7.1.4. Context Diagram (Level 0 Data Flow Diagram) 23
7.1.5. Level 1 Data Flow Diagram 25
7.1.6. Level 2 Data Flow Diagram 28
7.2. Class Model 26
7.3. State Model 30
8. To Be Determined List 27

1. Introduction

1.1. Purpose and Scope

This SRS document will describe what our project will be and how it will perform, by characterizing in detail the requirements, architecture and intended user cases. The SRS will characterize the entire system.

Many anomaly-based intrusion detection systems (AIDS) use traditional machine learning techniques, but there has not been extensive exploration of neural networks (NN) evolved using genetic algorithms (GA) as a tool for intrusion detection [3].

Our product will be AIDS software. Our project can be described in two distinct parts. We will refer to these two distinct parts as either training or testing. Both the training and testing of our software will be heavily dependent on our chosen dataset.

We selected the NSL-KDD dataset. This dataset is composed of network related metadata and packet information, including data representative of both normal network behaviors and malicious activity. The data will not be collected and provided to the system in a real-time environment. The intended result is to evolve a NN with the ability to identify genuine malicious activity within normal network traffic. Our software will have no mechanism to counter malicious actions. Instead, the resulting NN will output a classification of the input data as either normal or suspicious [2]. These flags accomplish the intrusion detection aspect of our project.

The training part will be how the anomaly-based classification mechanism is developed. The AIDS software will be developed using a NN evolved over several generations using a GA. We will be using the original HyperNEAT implementation, which is a well-established open-source GA implemented in C++. Our selected GA will only be modified to accommodate data inputs and outputs and/or to accommodate the integration of our NN with our aforementioned GA. The combination of HyperNEAT and our implemented NN will be further referred to as neuroevolution. The resulting neuroevolutionary behavior will allow us to achieve the anomaly-based classification mechanism of our proposed software. We will further diversify our GA for experimental learning by implementing three different fitness algorithms in three different software builds. Graphical comparisons will be made to evaluate the resulting performance of each unique fitness algorithm.

In the testing part we will be quantifying the behavior of our software using predefined performance metrics and the classification flags the system produces for each data entry contained within the dataset.

By the end of this project, we will have produced software that allows us to observe the potential accuracy and novelty of AIDS using neuroevolution. Therefore, we consider producing a neuroevolutionary behavior to be an essential and verifiable behavior of our project. Additionally, the program will be able to use the dataset's network data as an input to train a NN and as input to be classified in the testing phase of our project.

1.2. Intended Audience and Reading Suggestions

This SRS document will require a reasonable familiarity with the topic of neuroevolution, genetic algorithms, neural networks, intrusion detection systems and introductory understanding of computer networks and packet architectures.

We recommend reading about convolutional neural networks and their prior implementations using NEAT or HyperNEAT algorithms. This will give the reader a general idea about the current state of neuroevolution and what inspired this project. The reader can also familiarize themselves with general concepts in the intrusion detection field, especially anomaly-based intrusion detection systems achieved using artificial neural networks. ANNs are held under the category of supervised machine learning. There are several different implementations of ANNs, understanding any would be appropriate. The reader could benefit from understanding multiple possible paths to implementing an intrusion detection system and how it compares to the benefits of an ANN implementation. The reader may also want to briefly review possible performance metrics that could be used to evaluate intrusion detection systems. This includes accuracy, true positive rates, true negative rates, and others. The reader will also need an introductory knowledge of cyberattacks including denial of service, probes, overflows and brute force attacks. Since our neural network will be learning the context of a network by using network communications, it would benefit the reader to understand features like socket connections, hosts, IP addresses and network traffic.

1.3. Document Conventions

No notable notation conventions are currently being in the SRS. (October 29th, 2024)

1.4. Project References

[1] J. Gauci and K. O. Stanley, “A case study on the critical role of geometric regularity in machine learning,” National Conference on Artificial Intelligence, pp. 628–633, Jul. 2008.

[2] B. Su, R. Li, and H. Zhang, “Evolving Deep Convolutional Neural Network for Intrusion Detection Based on NEAT,” Oct. 2020, doi: https://doi.org/10.1109/wpmc50192.2020.9309451.

[3] K. Ansam et al, "Survey of intrusion detection systems: techniques, datasets and challenges," Cybersecurity, vol. 2, (1), 2019. Available: https://www.proquest.com/scholarly-journals/survey-intrusion-detection-systems-techniques/docview/2259210177/se-2. DOI: https://doi.org/10.1186/s42400-019-0038-7.

1.5. Definitions, Acronyms, and Abbreviations

1.5.1. Definitions

This section lists terms used in this document and their associated definitions.

See reference [3] for definitions table.

Table 1: <Definitions>

Term	Definition
Intrusion Detection System	A system designed to identify and respond to malware
Anomaly-based intrusion detection system	An IDS that detects malware through a normal-anomalous classification system
Machine Learning	A system creates a system of “knowledge” using pattern recognition, behavior prediction and large amounts of data
Supervised Learning	A subsection of machine learning where the inputted data is labeled to assist in development of a classification system
Genetic Algorithms	A system of data processing rules developed by imitating the behaviors of biological evolution
Artificial Neural Network	A system of data processing developed by imitation the neurons and synapses of the brain
Accuracy	The percentage of all those correctly predicted instances to all instances
HyperNEAT	A genetic algorithm derived from the NEAT algorithm that uses CPPN to generate connections between nodes in a multi-dimensional space
Malware	Software designed to steal, destroy or sabotage information and data
Data Label	A cell entry attached to a row in the database that acts as a classifying value for an event or action that occurs within a network or host
Default Fitness Algorithm	[2]
Novelty Search	
MAP-Elite	
Quality Diversity	

1.5.2. Acronyms

This section lists the acronyms used in this document and their associated definitions.

1.5.3. Table 2: Acronyms

Term	Definition
ACC	Accuracy Rate

AIDS	Anomaly-based Intrusion Detection System
ANN	Artificial Neural Network
BP	Backpropagation
CNN	Convolutional Neural Network
CPPN	Compositional Pattern Producing Networks
CR	Classification Rate
DDos	Distributed Denial-of-Service
DNN	Deep Neural Network
DoS	Denial of Service
DR	Detection Ratio
EA	Evolutionary Algorithm
FNR	False Negatives Rate
FPR	False Positive Rate
GA	Genetic Algorithm
GUI	Graphic User Interface
HIDS	Host Based Intrusion Detection System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
HyperNEAT	Hypercube-based NEAT
ICMP	Internet Control Message Protocol
IDS	Intrusion Detection Systems
IP	Internet Protocol
KDD 99	Knowledge Discovery and Data Original Dataset 1999 version
MAP-Elites	Multi-dimensional Archive of Phenotypic Elites
ML	Machine Learning
NEAT	NeuroEvolution of Augmented Topologies
NIDS	Network Intrusion Detection System
NN	Neural Network
NNTP	Network News Transfer Protocol
NS	Novelty Search
NSL KDD	Network Security Laboratory Knowledge Discovery and Data Dataset
QD	Quality Diversity
R2L	Root to Local
ROC	Receiver Operating Characteristic
SDD	System Design Document
SIDS	Signature-based Intrusion Detection System
STP	System Test Plan
TCP	Transmission Control Protocol
TPR	True Positive Rate
U2R	User to Root
UDP	User Datagram Protocol
UI	User Interface

1.5.4. Abbreviations

This section lists the abbreviations used in this document and their associated definitions.

Table 3: <Caption>

Term	Definition
e.g.	For example

2. General Description

2.1. Product Perspective

The product described in this SRS is a standalone product that's primary purpose is learning the strengths and the limitations of the neuroevolution process when applied to an intrusion detection scenario. There are many different implementations of the neuroevolution process, using different algorithms or languages. This project will utilize HyperNEAT, an extension of NEAT. NEAT stands for NeuroEvolution of Augmenting Topologies. It is a genetic algorithm that evolves artificial neural networks in both weight and structure to work towards building stronger and more fit solutions to the problem at hand. HyperNEAT utilizes compositional pattern producing networks (CPPNs) which allows for more complex connections. A CPPN produces ANNs based on a set of input data. These ANNs are different because they are created from a four-dimensional space in which the data points are placed. Each point becomes a neuron in the ANN which allows the networks themselves to gain more knowledge about their own placement in the geometry. The name HyperNEAT comes from this four-dimensional hypercube [1].

This product is loosely based on a previous project, “Evolving Deep Convolutional Neural Network for Intrusion Detection Based on NEAT” [2]. In this past project a team attempted to evolve a neural network to perform intrusion detection using NEAT. This current project seeks to employ HyperNEAT to attempt improvement from the original project. Testing will allow for further comparison between HyperNEAT and NEAT as their algorithms apply to intrusion detection.

2.2. Product Features

The goal of this product is to test the usefulness of neuroevolution when applied to an intrusion detection system (IDS). Neuroevolution utilizes evolutionary algorithms to continuously evolve artificial neural networks (ANNs). Neuroevolution strives for novelty in its evolution process, rather than optimization like many other machine learning techniques. Anomalous intrusion detection is done by analyzing large amounts of network traffic. During the analysis the system will flag activity it determines to be anomalous, or potentially dangerous. To develop an IDS, a database is fed through these neuroevolution algorithms, and the ANNs reproduce based on what is learned from said database. The database will be an open-source data set of network data containing both anomalous and normal activity. After the completion of the evolution process testing is done to determine the accuracy in anomalous activity detection and speed of execution for the created IDS.

Figure 1 features a Use Case diagram for the Intrusion Detection system. There is only one actor and 5 use cases. For this project we will refer to the actor as a researcher. The produced IDS system is not of extreme complexity due to the goals of this product. The focus is on the

neuroevolution process, rather than the complexity of the system produced by it. The primary uses of the system from the IDS focus are to intake a dataset from the researcher, run anomaly detection analysis of data sets, and view detection reports. The researcher will also have the capability to insert data for training and view performance reports for the neuroevolution focus of the project.

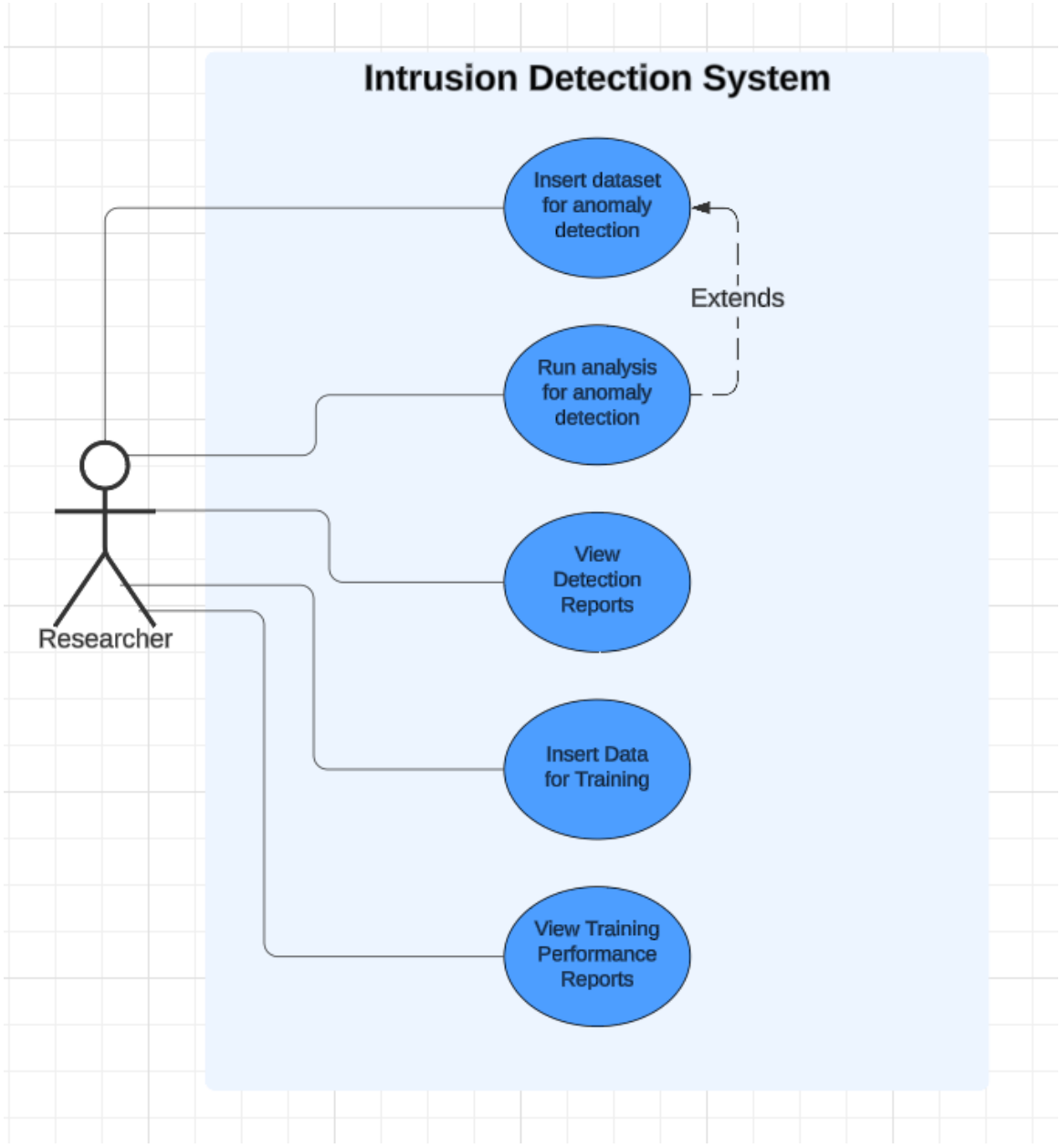


Figure 1. Use Case diagram of the intrusion detection system

2.3. User Classes and Characteristics

2.3.1. Actors

- Researcher
    - The researcher is the only actor in this system. They can use the system for intrusion detection or to monitor development and accuracy of the system. They have the ability to insert their own data, run analysis on said data, and view detection reports created from the analysis. They can also insert data for training, and view performance reports.

2.3.2. Use Cases

- Insert Testing Dataset
    - The researcher will upload a set of data containing network activity. This system will ensure that the data set has proper formatting to ensure it is possible for analysis to be performed.
- Run Analysis
    - The researcher will prompt the system to run analysis on the uploaded dataset. The system will then perform a scan for anomalies throughout the provided dataset.
- View Detection reports
    - The researcher will request the detection reports for an analysis that was performed. The system will compile said report and print it for the researcher.
- Insert Data for Training
    - The researcher will upload a data set into the system, and the system will ensure it has the ability to train based on said data.
- View Performance Reports
    - The researcher will request performance reports, and the system will compile said report and print it for the researcher.

2.3.3. Scenarios

Scenario 1: Insert Testing Dataset

Description: The researcher uploads a network activity dataset to the system.

Actors: Researcher

Precondition: The system has been launched

Trigger Condition: The researcher selects to upload dataset

Steps:

1. System prompts for researcher to upload file from pc
2. Researcher selects file to upload
3. System confirms file type is valid (ALT 1)
4. System reads format of data set to validate (ALT 2)
5. System prints message confirming upload
6. System returns to main menu

(ALT 1)

3. 1. System finds file type invalid
3. 2. System prints message stating upload failed due to file type
3. 3. Use case returns to step 1

(ALT 2)

4. 1. System finds data set format invalid
4. 2. System prints message stating data set formatting is invalid
4. 3. Use case returns to step 1

Scenario 2: Run Analysis

Description: The system performs analysis of uploaded network activity data set.

Actors: Researcher

Precondition: System has received a data set from user.

Trigger Condition: The researcher selects perform analysis action.

Steps:

1. System prompts researcher to select data set to perform analysis on
2. Researcher selects desired data set from selection available
3. System navigates to first line of network activity
4. System scans line of network activity
5. System finds activity normal (ALT 1)
6. System moves to next line
7. System finds next line empty (ALT 2)
8. System ends scanning process
9. System creates detection report containing anomaly count and all anomalous lines
10. System prints message to researcher stating scan completed

(ALT 1)

5. 1. System finds activity is anomalous
5. 2. System increases anomalous activity instance counter by 1
5. 3. System adds line number to array of line numbers deemed anomalous
5. 4. Resume Use Case at step 5

(ALT 2)

7. 1. System finds next line populated
7. 2. Resume Use Case at step 4

Scenario 3: View Detection Report

Description: The system displays detection report

Actors: Researcher

Precondition: System has received a data set from researcher and performed analysis

Trigger Condition: The researcher requests detection report

Steps:

1. System prompts researcher to select which data set it would like a report from
2. Researcher selects a data set
3. System displays report for researcher
4. Researcher closes report

Scenario 4: Insert Data for Training

Description: The researcher enters data for training

Actors: Researcher

Precondition: System is active

Trigger Condition: The researcher prompts system to begin training

Steps:

1. Researcher selects training network activity data set
2. System confirms file type is valid (ALT 1)
3. System runs performing training activity
4. System creates performance file
5. System adds training data to file
6. System alerts researcher of training completion

(ALT 1)

2. 1. System finds file type invalid
2. 2. System prints message stating upload failed due to file type
2. 3. Use case returns to step 1

Scenario 5: View Performance Report

Description: The system displays performance report

Actors: Researcher

Precondition: System has completed a training request

Trigger Condition: The researcher requests performance report

Steps:

1. System prompts researcher to select which training set it would like a report from
2. Researcher selects a data set
3. System displays report for researcher
4. Researcher closes report

2.4. General Constraints

A list of general constraints for the system.

I. The system will only flag activity it deems anomalous, it will not take action against flags.
II. The system will run on standard multi-core CPUs.
III. C++ will be the primary language used for the system with exceptions for any graphical techniques necessary.
IV. The system will utilize the existing neuroevolution algorithms and libraries of HyperNEAT for the evolutionary process.
V. The system will not have an overly complex GUI.
VI. The system will include extensive documentation regarding design, requirements, and testing.
VII. The testing process will be traceable and repeatable.

2.5. Operating Environment

For the system to properly operate the environment must be capable of executing C++ code, and capable of storing a data set for the product to analyze. The intensity of the neuroevolution process may increase in the future, however as of now the process should be able to execute on a computer that includes a CPU with the multi-threading ability.

The program is intended to run on a regular personal computer in a controlled environment comparable to an office workspace. The program should not be run on a personal computer in a location that is above room temperature, wet, bright or subject to long-term UV exposure.

The Operating system environment recommended for this project is Microsoft Windows 10 or newer. There is no other expected or associated hardware beyond a personal computer and any additional external hard drives deemed necessary.

2.6. User Documentation

GitHub ReadMe: Provides brief overview of project including implementation and tutorial for set up. Will include IEEE references to the HyperNEAT implementation.

System Design Document (SDS): A detailed breakdown of the product's implementation including a summary of the goals for the product, IEEE references, system architecture, and detailed design.

System Test Plan (STP): A document that includes all information regarding testing of the product. This includes test scope, test approach, test plan, traceability matrix and defect tracing, test cases, and more.

Engineering Journals: Detailed journals from each team member with detailed entries for each time the project was worked on. This includes team meetings, individual work, and submissions.

2.7. Assumptions and Dependencies

The development team has made the following assumptions:

- HyperNEAT will remain open-source and functioning during the project
    - HyperNEAT has the following dependencies
        - HyperNEAT depends on the TinyXMLDLL Library version 2.0,
        - JG Template Library (http://sourceforge.net/projects/jgtl/),
        - Boost c++ libraries (http://www.boost.org/)
        - WxWidgets *Only if building in GUI mode* (http://www.wxwidgets.org/).
        - HyperNEAT uses CMake 2.6 or later as build system
- NSL-KDD will remain open-source during the project

3. External Interface Requirements

3.1. User Interfaces

[REQ-301] The system shall provide UI elements to allow for dataset insertion for analysis

[REQ-302] The system shall provide UI elements to allow for dataset insertion for training

[REQ-303] The system shall provide UI elements to allow user to start evolution process

[REQ-304] The system shall provide UI elements to allow user to view analysis reports

[REQ-305] The system shall provide UI elements to allow user to view training reports

[REQ-306] All UI elements should be able to be accessed within 30 seconds of software boot.

[REQ-307] Error display messages must be displayed within 5 seconds of error.

Requirements and further details for the UI of this system are difficult to spell out as of Oct 29, 2024. This is due to the research nature of the project. The focus is more so on the process of developing the IDS rather than the UI systems that accompany it.

3.2. Hardware Interfaces

This system is entirely a software-based system. Other than the computer on which the software is run, there are no other external hardware interfaces necessary.

The software may benefit from the existence of USB ports, Cloud access, or other external hard drives for the storage of data sets that may be uploaded, however these interactions are not necessary for the system to run, as long as the dataset is contained on the personal computer and its conventionally accessible in the file explorer.

3.3. Software Interfaces

Required software as of Oct 29, 2024, for the reproduction of this project:

1. **Microsoft Windows** (This version or newer)
    a. Name: Windows 10 November 2021 Update
    b. Mnemonic: 21H2
    c. Specification Number: NA for Windows, refers to version number
    d. Version Number: 21H2
    e. Source: Microsoft's official documentation and update history pages.
2. **HyperNEAT**
    a. Name: Hypercube-based NEAT
    b. Mnemonic:
    c. Specification Number:

   d. Version Number: 4.0
   e. Source: https://github.com/MisterTea/HyperNEAT
3. **TinyXMLDLL Library version 2.0**
   a. Name: TinyXML
   b. Mnemonic: TinyXML
   c. Specification Number:
   d. Version Number: 2.6.2
   e. Source: https://sourceforge.net/projects/tinyxml/
4. **JG Template Library**
   a. Name: JG Template Library
   b. Mnemonic: JGTL
   c. Specification Number:
   d. Version Number: 1.6
   e. Source: http://sourceforge.net/projects/jgtl/
5. **Boost C++ libraries**
   a. Name: boost C++ source libraries
   b. Mnemonic: boost
   c. Specification Number:
   d. Version Number: 1.86.0
   e. Source: http://www.boost.org/
6. **WxWidgets**
   a. Name: wxWidgets Cross-Platform GUI Library
   b. Mnemonic: wxWidgets
   c. Specification Number:
   d. Version Number: 3.2.6
   e. Source: http://www.wxwidgets.org/
7. **CMake**
   a. Name: CMake
   b. Mnemonic: CMake
   c. Specification Number:
   d. Version Number: 3.31.0
   e. Source: https://cmake.org/

3.4. Communications Interfaces

This project is primarily self-contained. There are no communications interfaces within the system itself.

Users seeking to replicate our findings will need to access open-source content on the internet to acquire the HyperNEAT code, the code for the fitness algorithms and to access the cited NSL-KDD dataset used to test our project.

4. Behavioral Requirements

4.1. Same Class of User

[REQ-401] The system shall have one admin level of access privilege, where the user can provide the dataset input data and view the resulting performance and evolution of the neural network.

4.2. Related Real-world Objects

[REQ-402] The dataset of network activity imitates a real-time network containing both anomalous and normal internet traffic.

[REQ-403] The columns corresponding to each dataset entry shall represent the attributes of the real-world counterpart: a network communication within a network running real-time

[REQ-404] Network data within dataset shall imitate real life cyberattacks including Denial of Service (DoS), Probe, User to Root(U2R), and Remote to Local (R2L).

4.3. Stimulus

[REQ-405] The neuroevolutionary behavior will be initiated by the user.

[REQ-406] The system shall accept datasets input by the user.

[REQ-407] If the system cannot find the relevant dataset or is provided with an incorrect dataset the software will not run and print an error notify the user of the issue.

[REQ-408] The system shall provide detection reports upon request by user.

[REQ-409] The system shall provide performance reports upon request by user.

4.4. Related Features

[REQ-410] The program shall evolve a evolve a NN using the HyperNEAT GA

[REQ-411] The program shall take formatted data from a data set as an input for training

[REQ-412] The program shall take formatted data from a data set as an input for testing

[REQ-413] The program shall display the results of the testing phase in the form of a detection report displayed for the user in the UI

[REQ-414] The program shall display the results of the user activated neuroevolutionary process, in the form of nodal parameters displayed for the user on the UI

4.5. Functional

[REQ-415] If an entry in a dataset is not in the correct format, the program shall output and alert message and skip to the next entry.

[REQ-416] The program shall return an error and not proceed in testing or training if the database is not found

[REQ-417] The program shall return an error and not proceed if the database is not a readable format

[REQ-418] The program shall return an error, terminate dataset I/O reading and proceed with the first 999,999th entries as if this is the complete dataset, if the database's size exceeds 1 million entries

[REQ-419] The program shall return an error if the user enters a request to view the parameters of a node that does not exist (exceeds the generation and node ID numbering bounds of the system)

[REQ-420] The user shall specify to the program a training data set and a testing dataset.

[REQ-421] The program shall train across the number of neural network generations specified by the user.

[REQ-422] The program shall train a neural network using the user-specified dataset.

[REQ-423] The program shall evaluate the performance of single neural network using a user-defined fitness function.

[REQ-424] The program shall use the HyperNEAT algorithm to evolve the trained neural networks.

[REQ-425] The program shall produce flags labeling network traffic as anomalous activities

5. Non-behavioral Requirements

5.1. Performance Requirements

[REQ-501] The software will support a single terminal on the machine it is running.

[REQ-502] The software shall support dataset sizes of up to 1,000,000 entries for training.

[REQ-503] The software shall support dataset sizes of up to 1,000,000 entries for testing

[REQ-504] The neural network training shall process, at minimum,100 dataset entries points per second.

[REQ-505] The neural network testing shall process, at minimum, 25 dataset entries per second.

5.2. Safety Requirements

Our software is intended to be used for research/experimentation purposes. As such, it poses little to no risk of sensitive data loss or other personal/organizational damage and does not require any specific safeguards, actions, or secure design.

5.3. Qualitative Requirements

5.3.1. Availability

Our software will be available as long as the latest version compiles and runs correctly and the user meets the requirements for running the software on their machine. While the software may save experimentation results, there will be no mechanisms for recovering the state of the program should it terminate unexpectedly while running. Such mechanisms would add unnecessary complication and bloat to our software.

5.3.2. Security

Our software is intended to be used for research and experimentation purposes. As such, it poses little to no security risk to users. The software will not need any additional requirements to prevent malicious use.

5.3.3. Maintainability

The HyperNEAT implementation we are using forms most of the code for our project. This part of our software will remain relatively unchanged and should be altered as minimally as possible. The implementation makes use of older versions of existing libraries, such as zlib or JGTL. Our

software should only use the versions of these libraries packaged with our chosen implementation of HyperNEAT. For these reasons, most of the basic software should require little to no maintenance.

The primary part of our software that we will need to maintain is the code for our experiments. Our chosen HyperNEAT implementation is built such that the experiments are modular pieces that are distinct from the core functionality. As such, we can add, remove, and change our experiment files without affecting our software's core functionality.

5.3.4. Portability

[REQ-506] The software will run on the Windows 10 operating system.

[REQ-507] The software will run on the Windows 11 operating system.

[REQ-508] The software will be developed and built using the C++ programming language.

[REQ-509] The project will be developed using the Visual Studio 2022 IDE.

[REQ-510] The project shall be compiled with MSVC 14.3.

5.4. Design and Implementation Constraints

Our software is intended to be used for research and experimentation purposes within ERAU. There are no standards or regulations that apply to our software. As such, there are no design or implementation constraints or limitations on our software.

6. Other Requirements

6.1. Database Requirements

[REQ-600] The database shall be open-source with a citable content publisher.

[REG-601] The database shall have rows that represent a single event or action within the network.

[REQ-602] The database shall have columns that represent a unique characteristic that describes the event or action of the corresponding row.

[REQ-603] The database shall be divisible in such a way that two resulting halves of the dataset shall contain both instances of malicious actions and instances of normal or background behaviors within the network.

[REQ-604] The database shall contain instances of events or actions of malicious intent; for example, DDos attacks, malware injection, probing, U2R, and others.

[REQ-605] The database shall contain instances of events or actions that would be considered routine, normal, non-malicious and/or background operations of the network.

[REQ-606] The database shall have a unique column that acts as a label for supervised machine learning, where the label identifies the corresponding event or action as either malicious or normal, using a consistent convention

[REQ-607] The database shall have at least 100,000 unique network events or actions to facilitate the machine learning process

[REQ-608] The database shall be contained in a file format that can be read in standard file I/O reading operations for C++ programming; for example, the csv file format.

[REQ-609] The database shall be partitioned into a testing and training subset.

6.2. Operations

Our software will operate in a training and testing mode.

The UI is expected to be very limited in complexity and will only require used prompts to begin loading data and testing data. The user will also interact with the software to receive the parameters related to the evolved NN and/or to receive detection reports. Otherwise, the neuroevolution function, the generation of performance metrics and the generation of a detection report is an automated process with no required actions during its computation process.

There are no backup or recovery functions.

The user is expected to facilitate the initial connection of the program to the testing and training dataset. The user may also choose to record any of the resulting NN parameters, performance

metrics or detection reports onto paper by hand, for further documentation of their experimental findings. The program will not provide a function for recording and saving past experimental findings, it will merely print them for the user during that specific section.

7. Analysis Models

7.1. Data Flow Model

7.1.1. Data Sources

The data sources and their inputs to the system identified in the data flow model are as follows:

- *Researcher*

7.1.2. Data Sinks

The data sinks and their system outputs identified in the data flow model are as follows:

- *Researcher*

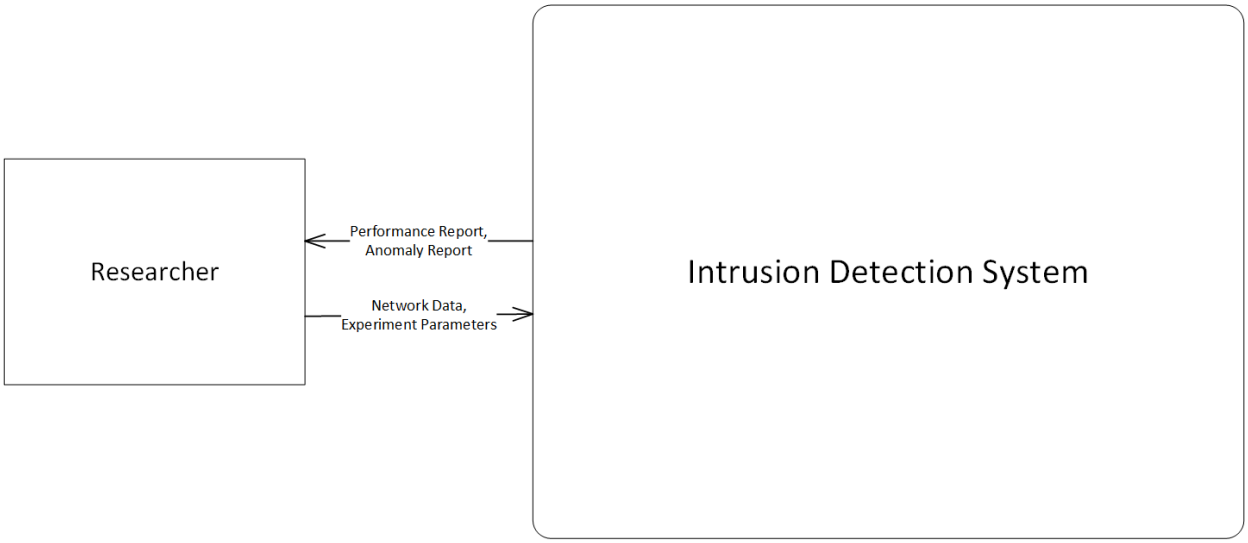
7.1.3. Data Dictionary

The data types are described in the data dictionary below. This section includes the name of the data type; a description of the contained data; how the data is structured; and the range of values.

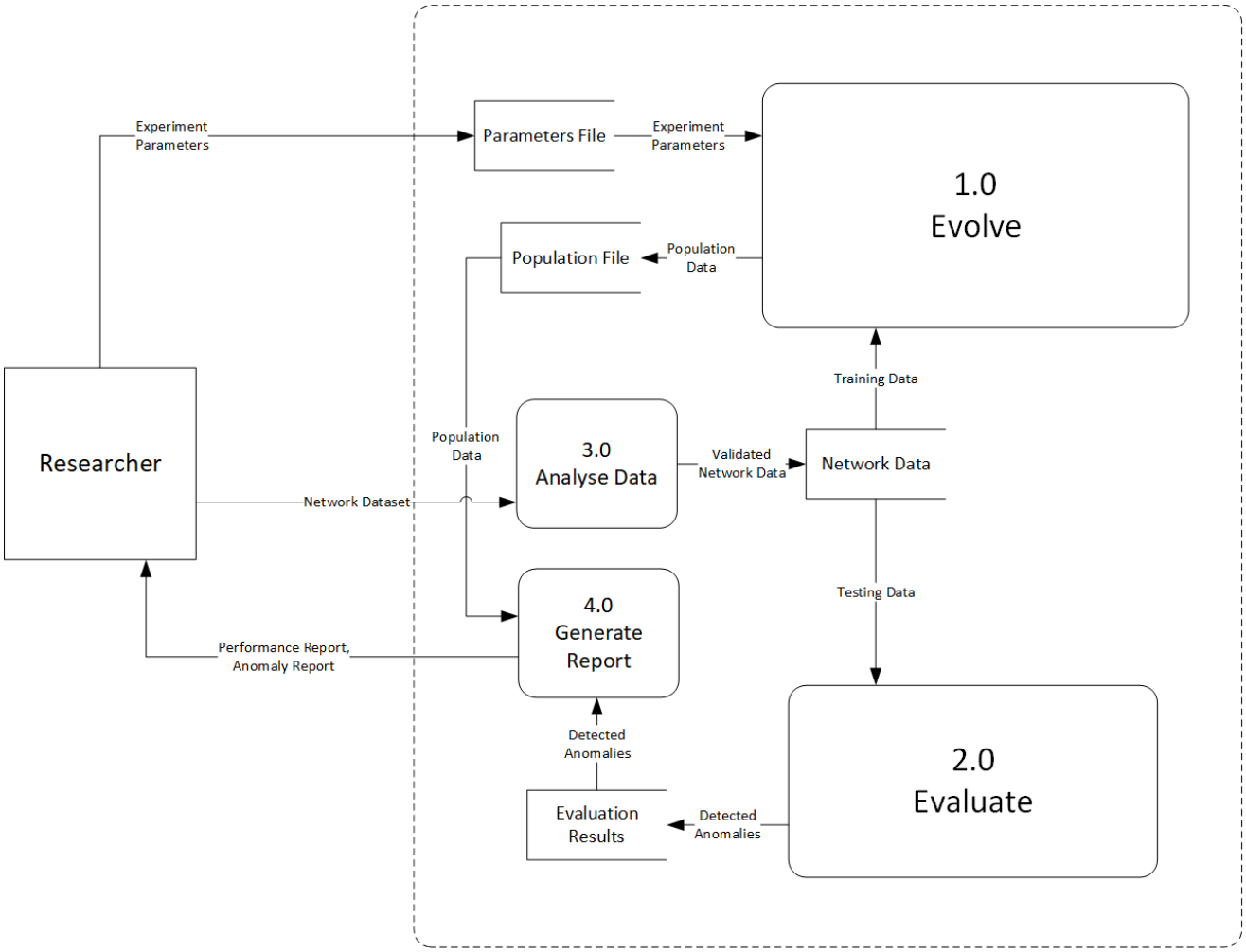
Name	***Description***	***Structure***	***Range***
Network Dataset	42 columns per entry corresponding to attributes like duration, protocol used, flag, source bytes, destination bytes, and more. The data points are too numerous to identify all of them by name and structure, but all 42 columns fall into one of three listed structure types. Labels are an important column in the dataset. An entry labeled with the standardized label “normal” indicates a background or non-malicious behavior. Any other string like “neptune” or “apache2” indicates some type of cyberattack. Labels are always a string.	Positive Integer Float String	0<
Validated Network Data	Process 3.0 (Analyse Data) ensures the Network Dataset is formatted in a manner the system can understand. Validated Network Dataset is simply the Network Dataset after being confirmed compatible.	Positive Integer Float String	0<

Training Data	A subset of the Network Dataset. Used by Process 1.0 (Evolve) to evaluate the fitness of the generated neural networks. The Training Data comes from the Network Dataset and therefore has consistent columns and data structures.	Positive Integer Float String	0<
Testing Data	A subset of the Network Dataset. Process 2.0 (Evaluate) takes the Testing Data, feeds it into the best neural network generated by Process 1.0 (Evolve), and identifies anomalous network traffic. The Training Data comes from the Network Dataset and therefore has consistent columns and data structures.	Positive Integer Float String	0<
Experiment Parameters	A set of multiple datapoints (population size, survival threshold, mutation probabilities, maximum number of generations, etc.) specific to the experiment being run.	Double	-1.0<
Population Data	A record of the individuals generated during an experiment sorted by generation, as well as the experiment parameters used; includes nodes, links between nodes, species IDs, and fitness.	Positive Integer Double String	0<
Evaluation Data	A record of the output of the Neural Network after being fed the Testing Data identifying anomalous activity and the network's confidence in the result.	Positive Integer Double String	0<
Performance Report	Formatted output containing the date and time the report was generated as well as select information from the Population Data.	String	N/A
Anomaly Report	Formatted output containing the date and time the report was generated as well as select information from the Evaluation Data.	String	N/A

7.1.4. Context Diagram (Level 0 Data Flow Diagram)


Researcher
Performance Report,
Anomaly Report
Network Data,
Experiment Parameters
Intrusion Detection System

7.1.5. Level 1 Data Flow Diagram



7.2. Class Model



8. To Be Determined List

<< Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure. >>